

1.2 Group Member Predictions Overview

Yankun Zhu | Individual prediction approach

I selected a target-specific LightGBM pipeline with signed target transformations, lag features and accounting-derived blends. The fixed candidate achieved mean out-of-fold (OOF) sMAPE of **16.5719%** on five shuffled folds with seed 314159. In an earlier common 80/20 benchmark, LightGBM achieved 29.3330%, the feedforward neural network 34.4984%, and the required mean baseline 136.1519%. These are two separate evaluation stages; none of these numbers is an official test-set score.

1.2.1 Data Prep

Data and targets. The training file contains 100,000 rows and 212 columns; the individual test file contains 50,000 rows and 203 columns. After excluding the identifier and all nine Q0 response columns, there are 202 predictors: 198 numeric and four categorical (industry, sector, financial currency and recommendation category). The responses are total assets, total liabilities, stockholders' equity, gross profit, cost of revenues, revenues, operating income, operating expenses and EBITDA. Q1-Q10 describe the ten preceding quarters. The two files have identical predictor names and order. **Id is never a predictor**; it is retained only for alignment and submission.

Missing values and encoding. The audit found no parsed missing cells, non-finite numeric values or missing targets. Literal category placeholders were retained as category levels: 1,344 training records in each of industry and sector, and 1,232 in each of financial currency and recommendation category. The pipeline nevertheless fits median numeric imputation and most-frequent categorical imputation on training rows only, then dense one-hot encoding with unseen validation/test categories ignored. This produced 332 encoded features in the original holdout benchmark. No validation/test statistics determine imputation, category vocabularies or scaling.

Scale and signs. Financial targets are highly skewed: total assets have a training median of about 0.823 billion versus a mean of 5.285 billion. Operating income and EBITDA are negative in 79.542% and 78.018% of training rows. Even assets include synthetic negative values. I retained these observations and the fractional fiscal-year-end indicators instead of dropping, winsorizing or binarizing them. Tree inputs remain on their original numeric scale; Ridge and the neural network additionally use arcsinh-transformed numeric predictors followed by training-only standardization.

Target transformation. For transformed regression, the scale is the training-only median absolute target, replaced by 1 only if it is nonpositive. I fit to $z = \operatorname{arcsinh}(y / s)$ and invert with $\hat{y} = s \sinh(\hat{z})$. This compresses long tails while preserving negative values and values near zero. Each CV fold estimates its own scale; the final fit estimates it from all training labels.

Data Prep: lag features and validation assumptions

Target-specific lag features. For gross profit, operating income and EBITDA, I append the same eleven row-wise features: Q1 value; Q1 minus Q2; Q1 sign; Q2 sign; the product of those signs; Q1-Q4 mean, median and population standard deviation; a Q1-Q4 linear slope; Q1-Q10 mean; and a Q1-Q10 slope. For a K-quarter slope I center the recency coordinates (K-1, ..., 0), then divide the weighted sum of values by the sum of squared centered coordinates. A positive slope means growth toward Q1. These features use only past-quarter values from the same row and require no target-based fitting.

Leakage and structure. The dictionary places Q0 at the latest reported quarter; the file contains neither a usable company grouping identifier nor varying row timestamps. There are no exact duplicate predictor rows or historical-feature rows. A shuffled row split is therefore a practical comparison for this supplied dataset, but cannot establish future-quarter or unseen-company generalization. Synthetic rows could still share an underlying company. Same-quarter metadata such as totalRevenue and ebitda were retained as provided features; their availability at a real forecasting date would need verification. The audit found correlation 0.9655 between totalRevenue and Q0 revenues. Excluding Id and Q0 labels prevents direct label inclusion, but does not resolve that deployment concern.

1.2.2 Method Selection

Common comparison. I used a fixed random 80/20 split with seed 42: 80,000 training rows and the same 20,000 validation rows for every initial model. I saved validation predictions with row index and Id to verify alignment. All preprocessing was fit on the 80,000 training rows; every method predicted all nine targets and was evaluated on the original response scale.

Metric. For target j , the assignment score is:

$$\text{sMAPE}_j = (100 / n) \sum_i |y_{ij} - \hat{y}_{ij}| / [0.5 (|y_{ij}| + |\hat{y}_{ij}|)]$$

The overall score is the unweighted average of the nine target scores; lower is better. I use zero error only when both actual and predicted values are zero. The denominator makes relative scale important: opposite signs give the maximum 200% error, and observations near zero can be difficult. The exact same scoring implementation was used throughout. Improvement over baseline is 100 times (baseline score minus model score) divided by baseline score.

Separation from test data. Test data were initially inspected for schema and missing values, then used for prediction only after the recipe was fixed. Test predictions and their distribution diagnostics did not select models, transformations or weights.

Method Selection: approaches considered

Naive baseline. Each response is predicted by its mean from the training portion, ignoring every predictor. This is the rubric's required reference rather than a fitted relationship between features and targets. Long tails pull the means away from a typical firm, and a constant sign cannot adapt to firm-level profit variation.

Non-deep-learning methods. Ridge provides a regularized linear reference. Random Forest averages bootstrap-grown trees; ExtraTrees randomizes tree thresholds more strongly. Both used multi-output regressors. HistGradientBoosting and LightGBM fit one boosted-tree regressor per target. These models are appropriate for tabular mixed-scale financial data because trees can capture nonlinear persistence and interactions without treating monetary scale as a distance. Ridge was retained as a diagnostic, despite its poor initial result.

Method	Fixed initial configuration
Ridge	Penalty 100; iterative least-squares solver; transformed/scaled numeric inputs
Random Forest / ExtraTrees	48 trees; maximum depth 14; minimum leaf size 8; feature fraction 0.7
HistGradientBoosting	80 iterations; 31 leaves; 128 bins; L2 penalty 1; early stopping disabled
LightGBM	120 trees; learning rate 0.08; 31 leaves; minimum leaf size 40; feature fraction 0.8

Deep learning: feedforward multi-output network. The network is a multilayer perceptron with **332 inputs, hidden layers of 128 and 64 ReLU units, and nine unrestricted linear outputs** (about 51,465 trainable parameters). Two moderate hidden layers give nonlinear capacity without making the initial experiment large. Shared hidden layers let related financial targets use common representations. A feedforward network fits the supplied tabular rows; the historical quarters are features, while an unrestricted output allows losses as well as profits. It was randomly initialized; no pretrained model or transfer learning was used.

The network uses the same imputation and category encoding as the other models. Numeric inputs are arcsinh-transformed and standardized. Each target is arcsinh-transformed using its training median absolute value, then centered and standardized using training-only moments. Predictions undo standardization and then apply sinh. Training uses AdamW (learning rate 0.001, weight decay 0.0001), SmoothL1 loss, batches of 2,048, and 12 epochs with seed 42. Target standardization balances the nine outputs and SmoothL1 reduces sensitivity to large errors in transformed space; neither directly minimizes sMAPE.

Initial tuning policy. These were fixed, modest configurations, not a large hyperparameter search. The network had one architecture and a fixed epoch budget, with no validation-based early stopping, convergence analysis or architecture search. The neural-network experiment therefore provides a limited initial benchmark. Its result evaluates this particular network pipeline and does not establish that all deep learning methods are inferior. The final submission contains the selected tree-based method, while this report documents the neural network that was actually considered.

Method Selection: common-split comparison results

Model	Mean sMAPE (%)	Improvement vs mean	Train time (s)	95% interval (%)
LightGBM (raw targets)	29.3330	78.46%	16.93	28.98-29.67
HistGradientBoosting	31.2930	77.02%	29.71	--
Random Forest	33.0163	75.75%	110.75	--
ExtraTrees	33.1464	75.65%	24.14	--
Feedforward network	34.4984	74.66%	7.46	34.12-34.86
Naive mean	136.1519	0.00%	0.00	135.56-136.75
Ridge	142.2592	-4.49%	10.42	--

Training times include preprocessing and fitting in the recorded local environment; they are illustrative rather than hardware-neutral comparisons. Intervals shown are conditional percentile intervals from 2,000 shared row-bootstrap resamples of the stored 20,000 validation predictions (seed 20261006); no model was retrained.

Target	Naive mean (%)	LightGBM (%)	Network (%)
Total assets	137.68	15.05	23.55
Total liabilities	141.11	17.00	24.11
Stockholders' equity	135.54	21.31	29.31
Gross profit	138.23	35.79	36.48
Cost of revenues	144.40	23.24	30.35
Revenues	140.81	19.91	24.01
Operating income	128.29	58.89	60.78
Operating expenses	127.71	12.46	19.12
EBITDA	131.60	60.35	62.78

Decision from the common benchmark. LightGBM had the lowest overall score (29.3330%) and was lower than the network for every target. The network still improved on the naive mean by 74.66%. The paired network-minus-LightGBM gap was **5.1654 percentage points**, with a 95% row-bootstrap interval of **4.9631-5.3693**. Random Forest was especially strong on assets, liabilities and equity, despite a worse overall mean, motivating target-specific rather than uniform model selection.

These intervals condition on the fitted models and exchangeable validation rows. They do not account for training randomness, model selection or hidden company clusters. The benchmark compares complete pipelines: input/target transforms differ by model, so it does not isolate architecture alone. The later optimized OOF score is reported separately, not as a new common-split neural-network comparison.

Method Selection: OOF refinement and controlled tuning

I next generated aligned five-fold OOF predictions (shuffle, seed 42) for Random Forest, LightGBM and HistGradientBoosting; each row was predicted without using that row to fit its model. Their base mean scores were 32.9352%, 29.3010% and 31.2856%, respectively. Q1 persistence, Q1-Q2/Q1-Q4 means, and a 0.60/0.25/0.10/0.05 recency average were also checked. Q1 persistence scored 47.3769% overall and 18.0507% for cost of revenues, showing useful persistence but poor robustness for profit signs.

RF/LightGBM and lag weights were tried target by target on these OOF predictions, then accounting-derived alternatives were compared. The resulting original recipe scored 21.2509% on the same selection OOF labels, an optimistic post-selection estimate. I froze it and reran it without changing choices or weights on seed 2026, obtaining 21.1944%. Both sets still contain the same training rows.

Transformation ablation. On the seed-2026 folds, raw and arcsinh LightGBM used identical base features, preprocessing and parameters. All nine targets improved in all five folds. Mean sMAPE fell from 29.1420% to 19.1005% for these single-model pipelines.

Target	Raw (%)	Arcsinh (%)
Total assets	14.93	4.59
Total liabilities	16.58	5.37
Stockholders' equity	21.34	9.29
Gross profit	35.91	23.40
Cost of revenues	22.61	11.61
Revenues	19.58	9.35
Operating income	59.26	51.56
Operating expenses	12.38	6.18
EBITDA	59.69	50.56

Hard-target representations. With the same target-specific lag features and seed-2026 folds, operating-income sMAPE was 51.7784% for direct arcsinh, 51.0429% for a modeled revenue margin, and **43.7901%** for sign plus magnitude. EBITDA was 50.3554%, 50.1255% and **42.9105%**, respectively; modeling EBITDA minus operating income gave 48.6407%. Margin and residual reconstruction used other targets' OOF predictions, not validation-row labels. Sign plus magnitude won in all five folds; its sign accuracy was 92.967% for operating income and 93.317% for EBITDA.

Limited search. Five configurations were compared only for arcsinh-beneficial targets: the 31-leaf/40-minimum baseline; 15/80; 63/20; 31/40 with 180 trees at learning rate 0.05; and the baseline with L1 loss. All others used 120 trees at rate 0.08 and feature fraction 0.8. A configuration needed at least 0.15 percentage-point gain and three of five fold wins. Candidate direct substitutions then needed at least 0.05 point improvement in the complete recipe and three fold wins, applied in a fixed one-pass target order. Selected changes actually won on all five development folds. Accounting/lag blend weights stayed frozen; seed 314159 was reserved before this selection.

Method Selection: fixed individual prediction recipe

The final approach uses six direct LightGBM regressions and two classifier/regressor pairs, producing eight direct financial outputs. Revenue is derived from gross profit and cost, so no independent revenue model is needed in the final output. All models use 120 trees, learning rate 0.08 and feature fraction 0.8; the final full-data random seed is 42. The table specifies the target-dependent differences.

Direct output	Model / loss	Leaves / min. leaf	Extra lags
Assets, liabilities, equity	Arcsinh regression; squared-error loss	63 / 20	No
Gross profit	Arcsinh regression; L1 loss	31 / 40	Yes
Cost of revenues	Arcsinh regression; L1 loss	31 / 40	No
Operating expenses	Arcsinh regression; squared-error loss	63 / 20	No
Operating income, EBITDA	Binary sign classifier + arcsinh magnitude regression	31 / 40 (both)	Yes

For each sign/magnitude pair, the classifier fits the indicator that y is positive. The regression fits $\text{arcsinh}(|y| / s)$, using the median absolute training target for s . The inverse-transformed magnitude is positive when the predicted positive probability is at least **0.5**, and negative otherwise. The threshold is fixed, not optimized on the test predictions.

Accounting-derived final outputs. Let D denote the unmodified direct predictions and let A, L, E, GP, C, OI, OE and B denote assets, liabilities, equity, gross profit, cost, operating income, operating expenses and EBITDA. Every expression below uses D values; already adjusted final outputs are never fed into another expression.

Final target	Frozen expression
Assets	D_A
Liabilities	$0.50 D_L + 0.50 (D_A - D_E)$
Equity	$D_A - D_L$
Gross profit / cost	D_{GP} / D_C , respectively
Revenues	$\max(0, D_{GP} + D_C)$
Operating income	$0.50 D_{OI} + 0.50 (D_{GP} - D_{OE})$
Operating expenses	$\max[0, 0.25 D_{OE} + 0.75 (D_{GP} - D_{OI})]$
EBITDA	D_B

The training identities $\text{assets} = \text{liabilities} + \text{equity}$ and $\text{operating income} = \text{gross profit} - \text{expenses}$ hold to floating-point precision. $\text{Revenue} = \text{gross profit} + \text{cost}$ is approximate, with identity sMAPE 0.7972%. This supports accounting-derived candidates, but noisy model predictions still require OOF evaluation. These are target-wise blends, not a joint constrained projection that guarantees every identity among final outputs. Nonnegative guards apply only to revenue and operating expenses, which are nonnegative in the training targets; other signed outputs are retained.

Method Selection: fixed-recipe validation and decision

The candidate scored 16.5413% on seed-2026 development folds. After the candidate choices were fixed, I retrained both it and the original recipe on the same new five-fold partition with seed 314159, with no reselection on that partition. The candidate's mean was **16.5719%**, versus **21.2064%** for the original recipe: a **4.6345 percentage-point** reduction (21.85% relative reduction). All nine targets improved in the pooled OOF comparison.

Target	Original recipe (%)	Fixed candidate (%)
Total assets	4.274	3.340
Total liabilities	9.486	5.522
Stockholders' equity	10.157	6.969
Gross profit	23.091	20.550
Cost of revenues	18.051	10.147
Revenues	13.774	8.355
Operating income	50.602	42.666
Operating expenses	11.015	8.514
EBITDA	50.408	43.085

Fold	Original (%)	Candidate (%)	Gain (points)
1	21.2183	16.6151	4.6032
2	21.1640	16.5043	4.6597
3	21.2057	16.4828	4.7230
4	21.2484	16.6768	4.5717
5	21.1957	16.5807	4.6149

The candidate improves all five folds; its sample fold standard deviation is **0.0798 points** and its range is 16.4828%-16.6768%. This is descriptive fold variability, not a confidence interval based on independent experiments. A new seed still uses the same 100,000 labeled rows previously used in model development; overlapping training folds and selection can bias performance estimates. It establishes stability to this repartition, not fully independent generalization.

Why I chose this approach. The common-split evidence favored boosted trees over the tested neural network; arcsinh, sign/magnitude decomposition and target-specific accounting blends then reduced the remaining financial-scale and sign errors consistently. Operating income (42.666%) and EBITDA (43.085%) remain the hardest targets. The neural network was not rerun on these final folds, so I do not interpret 16.5719% as a head-to-head final-CV comparison against it.

I refit the frozen method on all 100,000 training rows to produce 50,000 individual test predictions. The self-contained submission script reproduced the existing final CSV byte for byte, preserved Id order, and passed finite-value and output-schema checks. This report does not supply a group-method comparison or a leaderboard/test score.

Evidence, reproducibility and code appendix

Results above are drawn from the saved project artifacts, rather than new model fits performed for this report. The only added statistical calculation resamples stored holdout errors to describe uncertainty; it does not alter any model, threshold or weight.

Evidence	Saved artifact(s)
Data audit and target distributions	audit_outputs/summary.json; target_distributions.csv; same_quarter_metadata.csv
Common-split three-approach comparison	benchmark_outputs/comparison.csv; validation_truth.csv; validation_predictions/
Conditional paired uncertainty	individual_report_outputs/uncertainty_summary.json
OOF selection and accounting checks	oof_outputs/base_model_scores.csv; lag_scores.csv; identity_checks.csv; target_recipes.csv
Arcsinh / hard-target development	ablation_cv_2026/target_scores.csv; representation_cv_2026/
Frozen candidate and new-fold results	candidate_cv_314159/selected_substitutions.json; held_target_scores.csv; held_fold_scores.csv
Final prediction artifact	final_submission.csv; submission_code_output.csv (identical)

Repository: [Bumblebee0427/Deep-Learning-Financial-Performance-Prediction](#). The separately supplied code appendix contains the complete relevant source listings, including the evaluation metric, non-DL benchmark, neural network, OOF selection and final pipeline. The narrative report above contains no source-code listing.

Complete source listing	Appendix pages
evaluation_framework.py	A1-A6
benchmark_models.py	A7-A14
oof_models.py	A15-A18
hard_target_oof.py	A19-A22
analyze_oof.py	A23-A30
locked_recipe_cv.py	A31-A36
arcsinh_ablation_cv.py	A37-A38
hard_representations_cv.py	A39-A41
lightgbm_small_search.py	A42-A44
candidate_recipe_cv.py	A45-A49
submission_code.py	A50-A55
individual_report_outputs/report_uncertainty.py	A56-A57